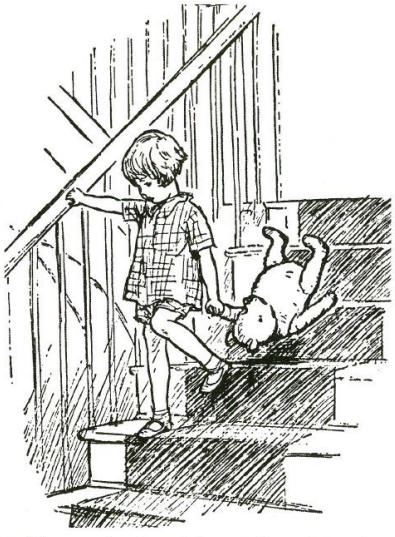
6. Traditional Process Models

"Software Development Methodologies"

Ricardo Filipe, ISEC (jcunha@isec.pt)

149

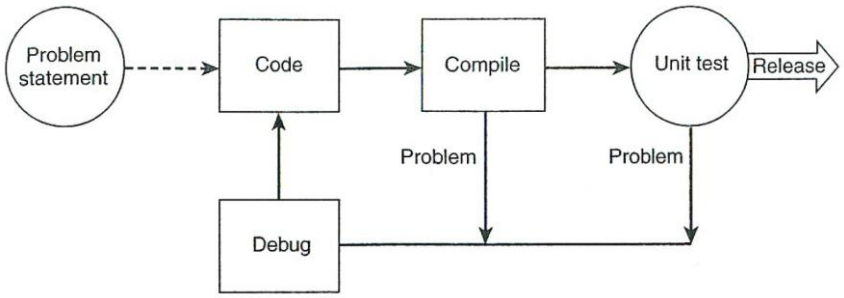


Here is Edward Bear, coming downstairs now, bump, bump, bump on the back of his head, behind Christopher Robin.
It is, as far as he knows, the only way of coming downstairs,
but sometimes he feels that there really is another way,
if only he could stop bumping for a moment and think of it.
And then he feels that perhaps there isn't.

A. A. Milne

150

Simple Code-and-fix process model



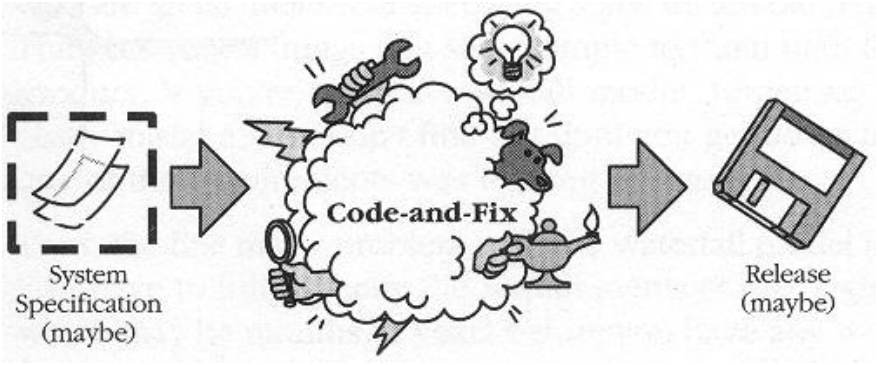
151

Code-and-fix

1. Discuss the software product
2. Write some code
3. Test to detect defects
4. Debug to find the cause of the defects
5. Correct the defects
6. If not done, go back to 1.

152

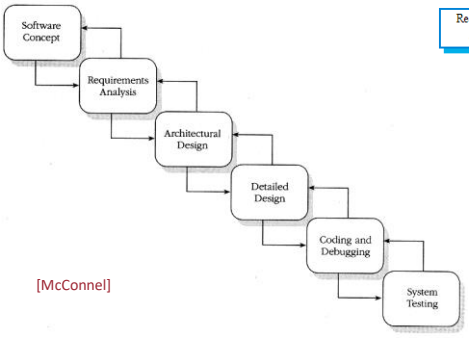
What Code-and-fix really is



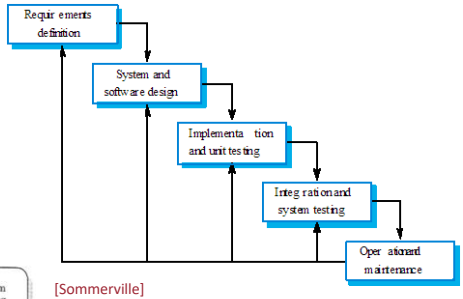
[From McConnel]

153

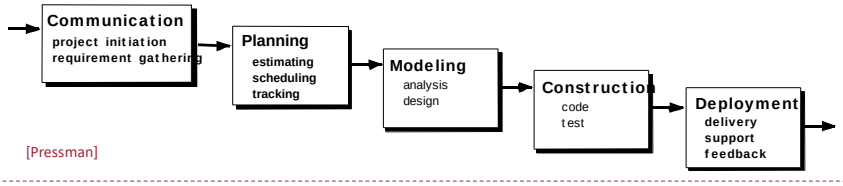
Waterfall



[McConnel]



[Sommerville]



[Pressman]

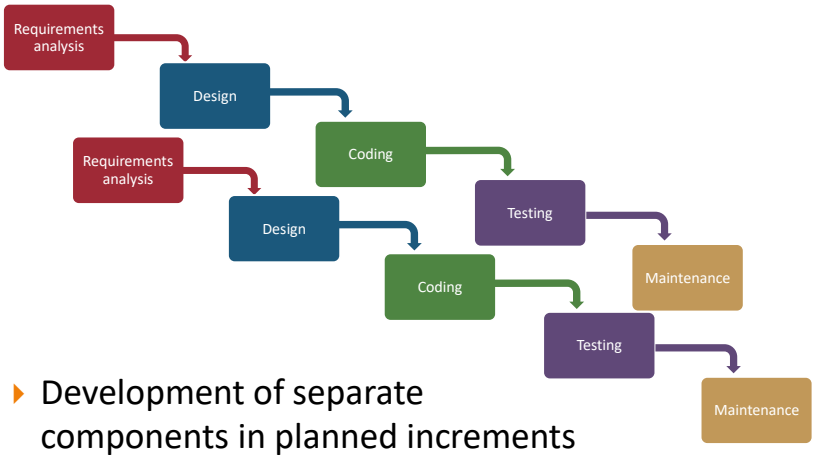
154

Waterfall

- Requirements must be specified in the first step
  - Difficult to accommodate the natural uncertainty at the beginning of the projects
- Models a systematic sequential approach
  - Real projects rarely follow the sequential flow
- The result of each phase is one or more approved documents
  - Document-driven approach – costly iterations
  - Prematurely freezing requirements may mean the system doesn't do what the user wants
- Works well when requirements are well understood, and technologies known

155

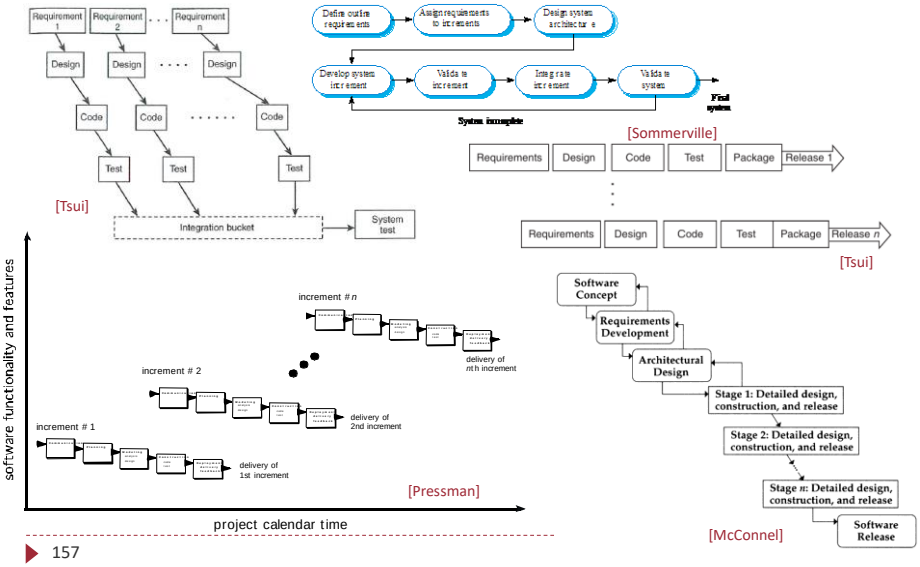
Incremental



- Development of separate components in planned increments
- Progress and problems get visible earlier

156

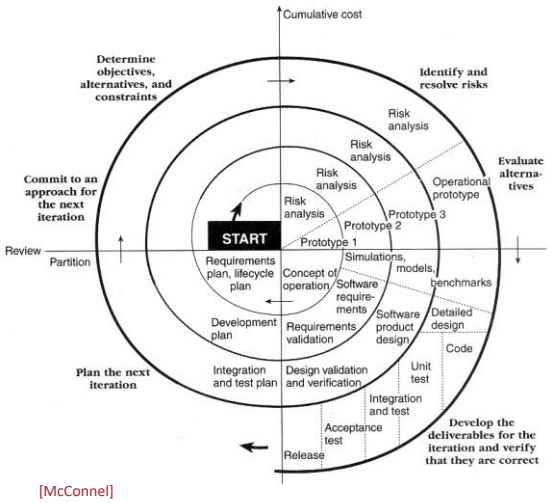
Incremental



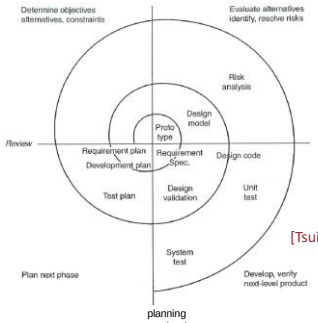
Incremental

- Development of separate components incrementally (and maybe iteratively)
  - Risk containment
  - Large products get decomposed into parts
  - Problems get visible earlier
  - Increments are planned
- Each increment may be released to the customer
  - Progress gets more visible to the team and client
  - First increment is the core product

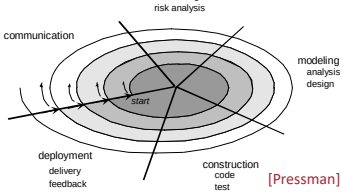
Spiral



[McConnel]



[Tsui]



[Pressman]

159

Spiral

A typical traversal through the four quadrants is as follows:

1. Identify the objectives, alternatives, or constraints for each cycle of the spiral.
2. Evaluate the alternatives relative to the objectives and constraints. In performing this step, many of the risks are identified and evaluated.
3. Depending on the amount of and type of identified risks, develop a prototype, more detailed evaluation, an evolutionary development, or some other step to further reduce the risk of achieving the identified objective. On the other hand, if the risk is substantially reduced, the next step may just be a task such as require-ments, design, or code.
4. Validate the achievement of the objective and plan for the next cycle.

160

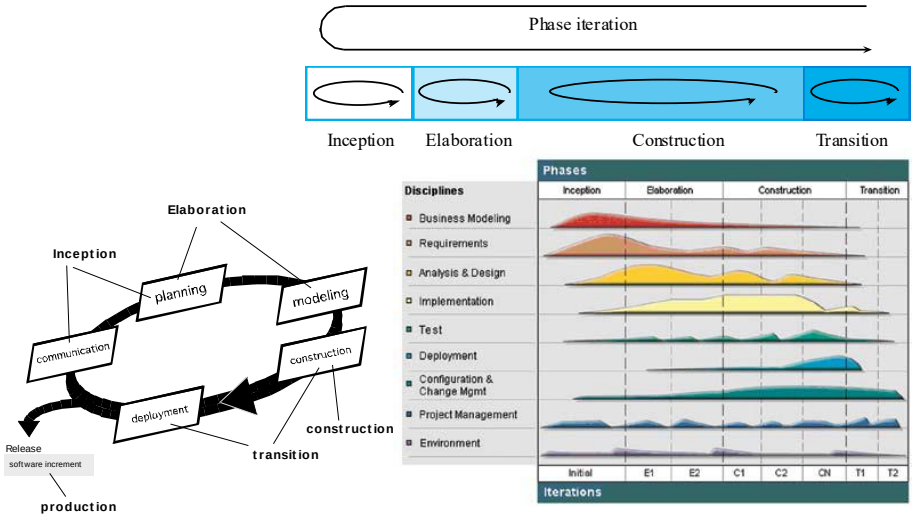
Spiral

- Each loop is a phase in the process
  - Loops are chosen depending on what's required
  - One release by the end of each loop (may be a model or a prototype)
- Spiral with 4 sectors
  - Objective setting
  - Risk assessment and reduction
  - Development and validation
  - Planning
- May be difficult to follow

Spiral

- Iterative and evolutionary model
  - Suited for products that evolve over time
- Risk-driven approach
  - Risks explicitly addressed and resolved
  - Model incorporates prototyping and modeling
  - Allows iterative and evolutionary approaches
  - The model does not preclude the rework of an early activity if a better alternative or a new risk is identified
- Ironically, it relies on risk assessment expertise. Not all SW engineers are trained or experienced in risk identification and analysis.

RUP – Rational Unified Process



RUP – Rational Unified Process

- Software process framework, developed by Rational Software Process
  - Now IBM RUP
- Closely aligned with the Unified Modeling Language (UML), based in three major concepts:
  - use-case driven
  - architecture-centric
  - iterative and incremental
- Three perspectives:
  - Dynamic perspective – showing phases and model over time
  - Static perspective – showing process activities
  - Good practices

RUP – Rational Unified Process

- Phases:
  - **Inception** – establish the business case for the system.
  - **Elaboration** – develop an understanding of the problem domain and the system architecture.
  - **Construction** – system design, programming and testing.
  - **Transition** – deploy the system in its operating environment
- Static Workflows (activities):
  - Business modelling
  - Requirements
  - Analysis and design
  - Implementation
  - Test
  - Deployment
  - Configuration and change management
  - Project management
  - Environment

RUP – Rational Unified Process

The inception phase may be viewed as the beginning stage when the product increment is still in an early stage of uncertainty. An initial idea is being developed during this phase. During the elaboration phase, the detailed use cases are being formulated, and the architecture and design are getting firmed up. The product increment is built, coded, and tested during the construction phase. Finally, the product increment is released to a small restricted group of users during the transition phase for further testing and correction. It is then released to the general public.

- Good practices:
  - Develop software iteratively – plan increments
  - Manage requirements – document requirements; trace requirements
  - Use component-based architectures – structure architecture
  - Visually model software – use UML
  - Verify software quality – ensure quality standards
  - Control changes to work products – manage changes

Bibliography

- **Tsui, Frank and Karam, Orlando.** *Essentials of Software Engineering.* Sudbury, MA : Jones and Bartlett Publishers, 2007 (Chapter 4)
- **Sommerville, Ian**. *Software Engineering - 8th ed*. Essex, England: Addison-Wesley, 2007 (Chapter 4)
- **Pressman, Roger S**. *Software Engineering: a practitioner's approach - 6th ed*. New York: McGraw-Hill, 2005 (Chapter 3)
- **IBM**. *IBM Rational Unified Process*. RAD10971-USEN-00, IBM Corporation Software Group, 2007
  ftp://ftp.software.ibm.com/software/rational/web/datasheets/RUP_DS.pdf